Instrumente și tehnici de bază în informatică
Curs 4

Introducere

- Interpretor pentru comenzi prin care interacționăm cu sistemul de operare
- Crearea de noi comenzi, automatizarea administrării sistemului
- Mai multe shelluri: `sh`, `bash`, `zsh`, `csh`; pot diferi în limbajul de scripting
- Pentru portabilitate: standardul POSIX
- Un script este compus din comenzi shell care pot fi: builtin sau din path

Introducere

Un prim exemplu:

```
#! /usr/bin/sh
echo "Hello world"
```

sau

```
#! /usr/bin/sh
read name
echo "Hello $name"
```

Shebang

- Pentru a rula un script shell `script.sh`, putem a apela un interpretor

  ```
  $ sh script.sh
  ```

- Dacă acesta începe cu o linie specială numită **shebang**, care specifică interpretorul prin care trebuie rulat scriptul, de forma

  ```
  #! /usr/bin/sh
  ```

  și are permisiuni de executare

  ```
  $ chmod +x script.sh
  ```

  îl putem executa direct cu sintaxa

  ```
  $ ./script.sh
  ```

- Shebang-ul nu este specific shell, putem aveam e.g. și

  ```
  #! /usr/bin/python
  ```

  pentru un program python.

Variabile

Putem asigna o valoare unei variabile folosind sintaxa `x=12` (**atenție:** fără spații; `x = 12` ar fi parsat drept comanda `x` cu argumentele =, 12)

```
$ x=12
$ echo $x
12
$ y=$x
$ echo $y
12
$ y=x
$ echo $y
x
```

Pentru a folosi valoarea unei variabile `x`, ne referim la aceasta cu sintaxa `$x`.
În mod similar, putem folosi rezultatul (din stdout) unei comenzi prin sintaxa `$(cmd)`

```
$ files=$(ls)
$ echo $files
```

Astfel, variabila `files` va conține stdout-ul comenzii `ls`.

Variabile de mediu

Asignarea variabilelor are efect doar în shell-ul curent. Pentru ca asignarea să rămână vizibilă în alte procese copil, o marcăm ca **variabilă de mediu**, folosind `export(1)`

```
$ my_var=12
$ ./print_my_var

$export my_var=12
$ ./print_my_var
12
```

Exemple: `$PATH`, `$HOME`
Cu `env(1)` putem observa toate variabilele de mediu

Expandare

- "Evaluarea" variabilelor și a comenzilor implică substituția, în esență textuală, a valorii acestora în script. Sintaxa `$var` sau `$(cmd)` (cu varianta echivalentă `` `cmd` ``) poate fi gândită ca declanșând expansiunea.
- Putem observa rezultatul expansiunilor în debugging, apelând shell-ul cu opțiunea `-x`.
- Quoting-ul controlează expansiunile:
  - `'aaa'` previne toate expansiunile

    ```
    $ echo 'echo $PATH'
    $PATH
    ```

  - `"aaa"` previne separarea suplimentară în cuvinte

    ```
    $ cat $(ls)
    ```

    va printa conținutul tuturor fișierelor din directorul curent

    ```
    $ cat "$(ls)"
    ```

    va încerca să printeze conținutul fișierelui al cărui nume este întreg output-ul lui `ls`

Variabile

Variabile speciale:

- `$?` codul de ieșire al ultime comenzi executate
- `$1, $2, $3, ...`, argumentele
- `$0` numele scriptului
- `$#` numărul de argumente
- `$@` toate argumentele (începând cu `$1`)
- `$*` toate argumentele (începând cu `$1`)

`$@` și `$*` se comportă diferit sub quoting:

- `"$@"` argumentele separate în cuvinte
- `"$*"` toate argumentele într-un singur cuvânt

Compunerea comenzilor

Reamintim că o comandă s-a executat cu succes dacă întoarce 0, și eșuează altfel.

- `cmd1 ; cmd2`

  Execută `cmd1` urmată de `cmd2` (indiferent de succesul lor). Codul de ieșire va fi codul ultimei instrucțiuni.
  Exemplu:

  ```
  $ gcc bad_program.c; ls . ; ls inexistent
  ```

  va afișa conținutul directorului curent și va întoarce 2

- `cmd1 && cmd2`

  Execută mai întâi `cmd1` și apoi `cmd2` dacă `cmd1` a avut succes.
  Exemplu:

  ```
  gcc main.c -o main && ./main
  ```

  va încerca să compileze `main` și îl va executa dacă compilarea a avut succes.

Compunerea comenzilor

- `cmd1 || cmd2`

  execută `cmd1` și apoi `cmd2` doar dacă `cmd1` a eșuat
  Exemplu:

```
$ gcc main.c -o main || echo "Compilation failed"

$ gcc main.c -o main && ./main || echo "Compilation failed"
```

Compunerea comenzilor

- `$ cmd1 | cmd2`

  execută `cmd1` și transmite `stdout`-ul lui `cmd1` ca `stdin` pentru `cmd2`

  ```
  $ ls | grep "main"
  ```

- Pentru a transmite `stdout`-ul lui `cmd1` ca **argumente** pentru `cmd2` putem folosi `xargs`

  ```
  $ cmd1 | xargs cmd2
  ```

  ```
  $ find . -name "*.txt" | cat
  $ find . -name "*.txt" | xargs cat
  ```

Instrucțiuni de control

Avem la dispoziție instrucțiunile de control tipice pentru limbajele imperative:

- Condiționale: `if`, `case`
- Repetitive: `for`, `while`, `until`, `break`, `continue`

Instrucțiuni condiționale

```
if cond; then
    cmd1
    ...
else
    cmd2
    ...
fi
```

Execută `cond` și, dacă aceasta are succes (în sensul codului de ieșire) execută blocul `then`, altfel execută blocul `else`.

Exemplu:

```
if gcc main.c -o main; then
    echo "Compilation successful"
else
    echo "Compilation failed"
fi
```

Comanda test

Comanda `test` poate fi folosită pentru a compara expresii aritmetice, stringuri, sau verifica proprietăți ale fișierelor.

Alternativ, avem sintaxa `[ expr ]` (**atenție:** spațiile sunt obligatorii)
Exemplu:

```
if [ "$1" -gt "$2" ]; then
    echo "$1 is greater than $2"
else
    echo "$1 is less than or equal to $2"
fi
```

Comanda test

Alte opțiuni:
Aritmetică:

- `[ a -eq b ]`: $a = b$
- `[ a -ne b ]`: $a \neq b$
- `[ a -ge b ]`: $a \geq b$
- ...

Stringuri:

- `[ s1 = s2 ]`: $s_1 = s_2$
- `[ -n s ]`: s are lungime nenulă
- ...

Operații logice:

- `[ ! e1 ]`: negație
- `[ e1 -a e2 ]`: conjuncție
- `[ e1 -o e2 ]`: disjuncție

Comanda test

Fișiere:

- `[ -e file ]`: `file` există
- `[ -d file]`: `file` există și este director
- `[ -f file ]`: `file` există și este fișier normal

Exemplu:

```
if [ -d "$1" ]; then
    echo "$1" is a directory
elif [ -f "$1" ]; then
    echo "$1" is a regular file
elif [ -e "$1" ]; then
    echo "$1" is neither a directory \
    nor a regular file
else
    echo "$1" does not exist
fi
```

Vedeți `man 1 test` pentru toate opțiunile.

Comanda test

Exemplu:

```
length=$1
file=$2
if [ "$length" -gt "$(cat $file | wc -l)" ]; then
    echo "$file" has at most "$length" lines
fi
```

Instrucțiuni condiționale

Putem facem pattern matching cu instrucțiunea `case`

```
case expr in
    pattern1)
        cmd1
        ...
        ;;
    pattern2)
        cmd2
        ...
        ;;
    ...
    *)
        cmdN
        ...
        ;;
esac
```

Instrucțiuni condiționale

Exemplu:

```
case "$filename" in
  *.c|*.cpp)
    echo "$filename is a C/C++ source file."
    ;;
  *.sh)
    echo "$filename is a shell script."
    ;;
  *)
    echo "$filename is some other file."
    ;;
esac
```

Instrucțiuni repetitive

```
for item in item1 item2 ... itemN; do
    cmd1
    cmd2
    ...
done
```

substituie `item` pe rând cu `item1`, `item2`, ..., `itemN` și execută blocul `do` pentru fiecare instanțiere.

Exemplu:

```
for arg in "$@"; do
    echo "$arg"
done
```

Exemplu:

```
for file in *.txt; do
    rm $file
done
```

Acesta funcționează deoarece `*.txt` se expandează la numele tuturor fișierelor `.txt` din directorul curent, sub forma `file1.txt file2.txt ...`

Instrucțiuni repetitive

Exemplu:

```
dir1="$1"
dir2="$2"
for file in *.c; do
    case "$file" in
        1*) mv "$file" "$dir1"/ ;;
        2*) mv "$file" "$dir2"/ ;;
        *)  ;;
    esac
done
```

Instrucțiuni repetitive

```
while cond; do
    cmd1
    cmd2
    ...
done
```

Exemplu:

```
read file
while [ ! -e "$file" ]; do
    echo "The file $file does not exist"
    read file
done
```

Instrucțiuni repetitive

Comanda `shift` shiftează cu o poziție la stânga toate argumentele.
Obținem astfel încă un mod de a itera argumentele

```
while [ "$#" -gt "0" ]; do
    echo "$1"
    shift
done
```

Funcții

Putem declara funcții cu sintaxa

```
function_name(){
    body
}
```

Spre deosebire de alte limbaje:

- argumentele nu sunt specificate în antentul funcției, ci sunt accesate cu `$1`, `$2`, ... (care vor fi diferite în contextul local al funcției)
- comanda `return` setează codul de ieșire
- folosim stdout (`echo`) pentru a întoarce alte valori
- nu avem variabile scoped

Funcții

Exemplu:

```
file_exists(){
    if [ -e "$1" ]; then
        return 0
    else
        return 1
}

read file
if file_exists $file; then
    echo "The file $file exists"
fi
```

Funcții

Exemplu:

```
count_lines(){
    if [ -f "$1" ]; then
        echo "$(cat $1 | wc -l)"
        return 0
    else
        return 1
    fi
}

for arg in "$@"; do
    lines="$(count_lines $arg)"
    if [ $? -eq 0 ]; then
        echo "$arg has $lines lines"
    fi
done
```

Funcții

Cum am spus, variabilele nu sunt scoped, de exemplu

```
func(){
    a=1
}
func
echo $a
```

va afișa `1`.